Universidad Autónoma de Madrid

Departamento de Informática

Sistemas Informáticos II
Práctica - 1

Roberto Marabini y Daniel Hernández Lobato

Registro de Cambios

Versión[1]	**Fecha**	**Autor**	**Descripción**
1.0	10.12.2024	RM	Primera versión.
2.0	6.10.2025	DHL	Segunda versión.

[1]La asignación de versiones se realizan mediante 2 números $X.Y$. Cambios en Y indican aclaraciones, descripciones más detalladas de algún punto o traducciones. Cambios en X indican modificaciones más profundas que o bien varían el material suministrado o el contenido de la práctica.

Índice

1. Objetivos **4**

2. Descripción de la Aplicación **4**
2.1. Propósito 4
2.2. Componentes 5
2.3. Flujo General 8

3. Instrucciones para Configuración y Despliegue **9**
3.1. Requisitos Previos 10
3.2. Estructura de directorios 11
3.3. Creación y Configuración del proyecto 11
3.4. Endpoints 14

4. Despliegue de la aplicación en las máquinas virtuales **15**
4.1. Requisitos Previos 15

5. Introducción a la API REST **18**
5.1. Características de una API REST 19
5.2. Implementación del API REST usando *Django* 19
5.3. Implementación del Frontend (Cliente) del API REST usando *Django* 24

6. Criterios de evaluación **26**

A. Configuración de Django **28**

B. Configuración de un Repositorio Git Remoto Usando SSH **28**
B.1. Crear un Repositorio Git Bare en el Equipo Remoto **VM2** 28
B.2. Conectar el Repositorio Local al Remoto 29
B.3. Actualizar Archivos Automáticamente en el Equipo Remoto 29
B.4. Realizar un Push Desde el Repositorio Local 30

C. Configuración de gunicorn **31**

C.1. Crear un Archivo de Configuración para el Servicio Gunicorn 31

C.2. Explicación de los Parámetros Importantes 32

C.3. Recargar `systemd` y Activar **gunicorn** 32

D. Script que lee la tabla tarjeta **33**

1. Objetivos

En el ámbito de la informática, la interacción eficiente entre clientes y servidores es un aspecto fundamental para el diseño de aplicaciones distribuidas. La selección de la tecnología adecuada para esta comunicación puede impactar directamente en el rendimiento, la escalabilidad y la latencia del sistema. Por ello, evaluar y comparar diferentes enfoques de comunicación es una tarea crucial para tomar decisiones informadas en el desarrollo de software.

El objetivo de las prácticas de esta asignatura es medir y analizar la eficiencia de la comunicación entre clientes y servidores utilizando diversas soluciones ampliamente empleadas en la industria, tales como:

- **API REST**: basada en el intercambio de mensajes HTTP, ampliamente adoptada por su simplicidad y compatibilidad.

En esta primera práctica nos centraremos en la implementación de una API REST y su despliegue en las maquinas virtuales.

2. Descripción de la Aplicación

En esta práctica vamos a partir de una aplicación de ejemplo, llamda **visaApp**, que tendrá que ser modificada para crear una aplicación web distribuida basada en una API REST.

La aplicación web implementada es un sistema de registro de pagos electrónicos diseñado para crear/borrar/listar pagos. Su propósito principal es garantizar la integridad del proceso de pago mediante la verificación de la tarjeta de la persona que paga y el control del registro de pagos. A continuación, se detallan sus componentes principales:

2.1. Propósito

El sistema tiene los siguientes objetivos:

- Garantizar que la tarjeta introducida esté registrada en el la base datos antes de que pueda registrar un pago.

- Asegurarse de que cada persona pueda registrar un único pago con el mismo idientificador de transacción por comercio.
- Proveer herramientas para registrar, eliminar y consultar pagos.

2.2. Componentes

Modelo de datos (models.py)

El proyecto utiliza dos modelos llamados **tarjeta** y **pago** los cuales se describen a continuación. La figura 1 muestra el diagrama relacional de la aplicación.

- **Modelo Tarjeta**:
  - Representa a las tarjetas habilitadas para pagar.
  - Principales campos:
    - `numero`: Identificador único de la tarjeta (clave primaria) del tipo WWWW XXXX YYYY ZZZZ.
    - `nombre`: Nombre completo del titular.
    - `fechaCaducidad`: Fecha de caducidad de la tarjeta del tipo MM/AA.
    - `codigoAutorizacion`: Código único de autorización del tipo XYZ.
- **Modelo Pago**:
  - Representa un pago emitido por un comercio.
  - Principales campos:
    - `id`: identificador (clave primaria)
    - `idComercio`: Id del comercio donde se registra el pago.
    - `idTransaccion`: Id de transacción asociada a ese comercio.
    - `importe`: Importe pagado en euros.
    - `tarjeta`: Relación con el modelo `Tarjeta`, representando al pagador.
    - `marcaTiempo`: Fecha y hora del registro del pago.
    - `codigoRespuesta`: Estado del pago (e.g., éxito o error).

- Restricciones implementadas en la base de datos:
  - No se puede emitir más de un pago con el mismo `idComercio` e `idTransaccion`.

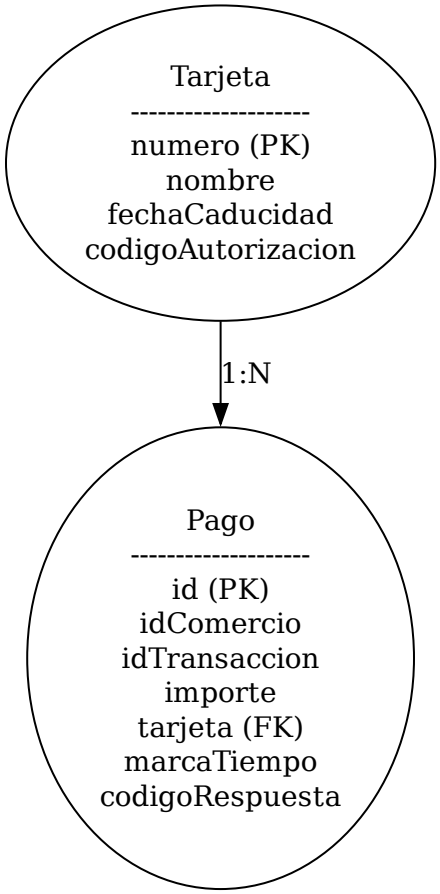

Figura 1: Diagrama relacional del modelo de datos. **PK** y **FK** significan clave primaria y clave extranjera respectivamente.

Lógica de la aplicación (views.py y pagoDB.py)

A continuación describimos las funciones que contienen la lógica de la aplicación base entregada al alumno.

- **aportarinfo_tarjeta**:
  - Permite autenticar a un usuario comprobando su registro en la tabla de tarjetas autorizadas.
  - Almacena el `numero` de la tarjeta en una variable de sesión llamda `numeroTarjeta` para su uso posterior en el método **aportarinfo_pago**.
- **aportarinfo_pago**:
  - Maneja el registro de pago.
  - Verifica que el pagador esté autenticado mediante el `numeroTarjeta` almacenado en una variable de sesión.
  - Registra el pago utilizando la función `registrar_pago`.
- **testbd**:
  - Integra la funcionalidad de validar una tarjeta en la tabla de tarjetas (aportarinfo_tarjeta) y el registro de pagos (aportarinfo_pago).
  - Verifica que la tarjeta esté en la tabla de tarjetas antes de registrar el pago.
- **delpago**:
  - Permite eliminar un pago identificado por un ID único.
- **getpagos**:
  - Recupera los pagos registrados en un comercio.

Las funciones descritas se implementan en **views.py** y están disponibles al público. Estas son responsables de procesar los formularios con los datos recibidos y generar las páginas web que se devuelven al navegador. Internamente, delegan en diversos métodos definidos en **pagoDB.py**, los cuales se encargan del acceso a la base de datos.

2.3. Flujo General

1. **Verificación de la Tarjeta**: El usuario ingresa su `numero` de tarjeta y otros datos mediante un formulario. Si está registrado en la base de datos, su numero de tarjeta se guarda en una variable de sesión.

2. **Registro de Pago**: El usuario selecciona un importe a pagar un id de comercio asociado y un id de transacción y registra el pago. El sistema verifica que no existan duplicados en la lista de pagos (mismo idComercio e idTransaccion). Esto es, que la misma paresona no ha pagado dos veces en el mismo comercio con el mismo idTransaccion.

3. **Gestión Adicional**: Los pagos pueden ser eliminados o consultados para comercios específicos.

La figura 2 ilustra el funcionamiento de la aplicación de partida **P1-base** y el flujo de información.

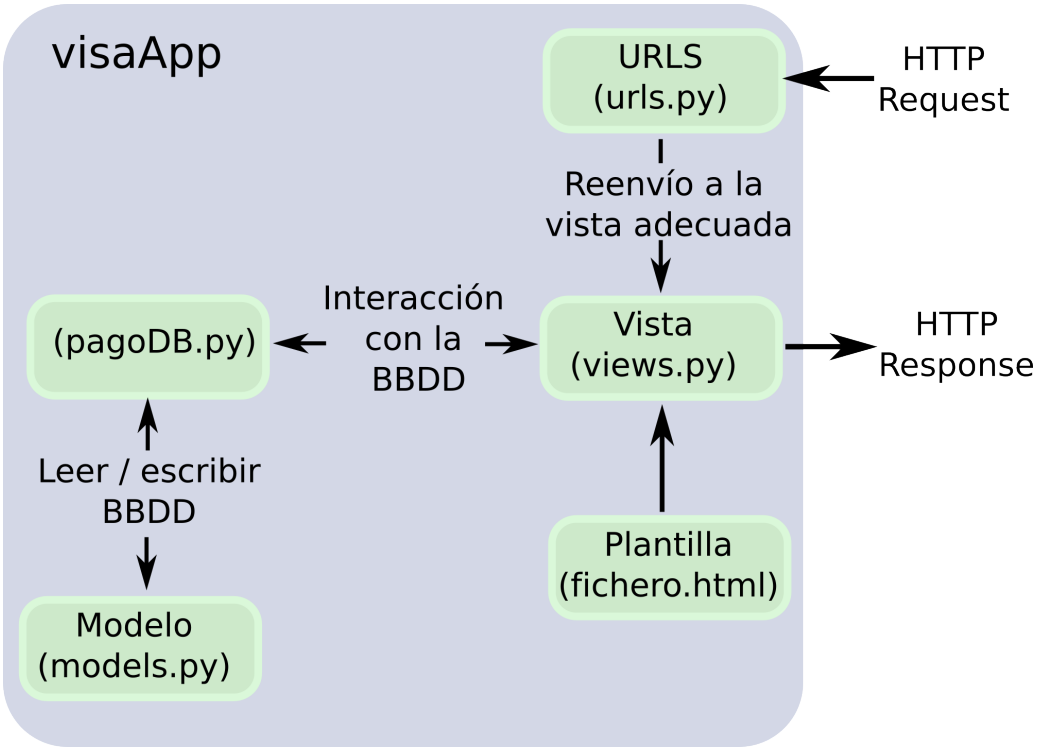

Figura 2: Descripción esquemática del flujo de información de la aplicación **P1-base**.

3. Instrucciones para Configuración y Despliegue

En esta sección describimos como desplegar el proyecto **P1-base** en el entorno de los laboratorios. Tal y como se muestra en la figura 3 serán necesarias dos maquinas virtuales a las que llamaremos **VM1** y **VM2** respectivamente. En **VM1** se desplegará la base de datos y en **VM2** la aplicación de *Django*. Si la dirección IP del ordenador **host** es XX.YY.ZZ.II la base de datos debería estar accesible en XX.YY.ZZ.II:15432 y el servidor de *Django* en XX.YY.ZZ.II:28000. En este documento llamaremos **host** al ordenador donde se ejecutan las máquinas virtuales. Ambas máquinas virtuales son accesibles por ssh en las direcciones XX.YY.ZZ.II:12022 (**VM1**) y XX.YY.ZZ.II:22022 (**VM2**). (Recuérdese que diversos puertos de la maquina **host** han sido redirigidos (port forwarding) a diversos servicios ejecutados en las máquinas virtuales).

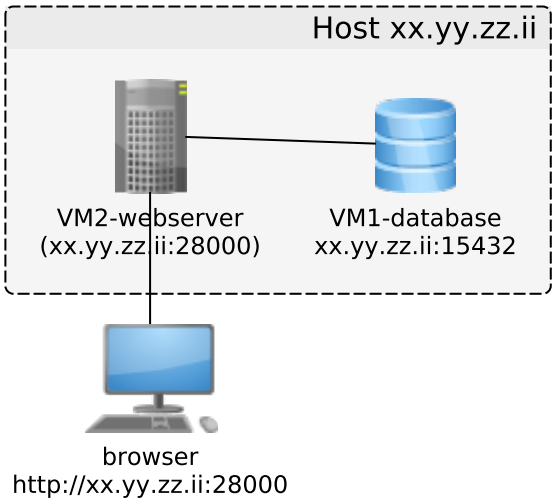

Figura 3: Descripción esquemática del despliegue del proyecto **P1-base**.

3.1. Requisitos Previos

Aunque queda a vuesra elección, recomendamos desarrollar y desplegar el proyecto localmente (en la máquina **host**) usando el servidor de bases de datos instalado en **VM1**. Una vez que la aplicación web funcione se desplegaría en su totalidad en las máquinas virtuales. En las máquinas virtuales ya se encuentra instalado el entorno pero ese no es el caso para el ordenador principal (**host**). El primer paso a realizar sería clonar el proyecto **P1-base** y conseguir que funcione localmente.

Antes de ejecutar la aplicación localmente, asegúrese de cumplir con los siguientes requisitos en el **host**:

- Tener instalado **Python** (versión 3.12).
- Tener un entorno virtual configurado (`venv` o equivalente).
- Haber instalado las dependencias descritas en el fichero **requirements.txt**.

3.2. Estructura de directorios

En la siguiente sección os solicitaremos que realicéis una copia (`git clone`) del repositorio `https://github.com/rmarabini/si2_alumnos.git` el cual contiene el proyecto **P1-base** junto con una colección de directorios donde se irán implementando el resto de las prácticas. La estructura de directorios es:

```
si2_alumnos
|
+-- P1-base
| +-- env
| +-- manage.py
| +-- requirements.txt
| +-- visaApp
| +-- visaSite
+-- P1-rpc-frontend
| +-- visaAppRPCFrontend
+-- P1-rpc-backend
| +-- visaAppRPCBackend
+-- P1-ws-frontend
| +-- visaAppWSFrontend
+-- P1-ws-backend
  +-- visaAppWSBackend
```

El proyecto **P1-base** sólo necesita pequeños ajustes pero los otros cuatro proyectos restantes (p.e. **P1-ws-backend**) deben ser implementados en su totalidad.

3.3. Creación y Configuración del proyecto

1. **Duplica el proyecto en *github***:

```
# Clonar el repositorio original
git clone git@github.com:rmarabini/si2_alumnos.git
```

```
# Cambiar al directorio del repositorio clonado
cd si2_alumnos

# Renombrar el repositorio remoto original
git remote rename origin old-origin

# crea un nuevo repositorio PRIVADO en github
# supongamos que se llama tu_nuevo_repositorio

# Agregar el nuevo repositorio como remoto
git remote add origin git@github.com:tu_usuario/tu_nuevo_repositorio.git

# Empujar todas las ramas al nuevo repositorio
git push -u origin --all
```

2. **Configurar el Entorno Virtual** en el **host**.

```
python3 -m venv venv
source venv/bin/activate
pip install -r P1-base/requirements.txt
```

3. **Configurar la Base de Datos**

   - El código asume que existe un fichero llamado **env** en la raíz del proyecto en el que se ha definido la variable DATABASE_SERVER_URL:

```
# env
DATABASE_SERVER_URL =
    ↪ 'postgres://alumnodb:alumnodb@localhost:XXXX/
    ↪ si2db'
```

Donde XXXX será 15432, si la base de datos se encuentra en **VM1**. Recordad que es necesario crear la base de datos "si2db" en la **VM1** para que el acceso a la base de datos sea correcto.

4. **Ejecutar Migraciones**

```
# en el directorio P1-base
python manage.py makemigrations
python manage.py migrate
```

5. **Crear un Superusuario (Opcional)**

```
python manage.py createsuperuser
```

El superusuario os da acceso al interfaz de administracion. Si queréis usarlo no os olvidéis de inicializar el fichero **admin.py** adecuadamente.

Listado 1: Listado del fichero **admin.py** mostrando como incorporar los modelos **pago** y **tarjeta** al sistema de administración de *Django*

```
# admin.py
from django.contrib import admin
from .models import Tarjeta, Pago

# Register Tarjeta model
@admin.register(Tarjeta)
class TarjetaAdmin(admin.ModelAdmin):
    pass

# Register Voto model
@admin.register(Pago)
class PagoAdmin(admin.ModelAdmin):
    pass
```

Código disponible en este enlace:

6. **Poblad la base de datos** con los datos de censo.

```
python manage.py populate
```

7. **Probar Localmente la Aplicación en el PC del Laboratorio**

```
python manage.py runserver
```

Accede a `http://127.0.0.1:8000/visaApp/nombre_endpoint` para verificar que la aplicación funcione correctamente. Los **endpoints** definidos se describen a continuación. Recordad que en los ordenadores de los laboratorios el puerto 8000 no está disponible y debéis usar otro puerto como pueda ser el 8001 (`python manage.py runserver 8001`).

3.4. Endpoints

El fichero `urls.py` define los **endpoints** que permiten interactuar con las funcionalidades principales de la aplicación. A continuación, se detallan algunos de ellos:

1. `tarjeta/`: utilizado para manejar la validación de los datos de la tarjeta. Es funcionalmente equivalente al endpoint raíz (`/`).

2. `pago/`: Permite a los usuarios cuya tarjeta ha sido validada (cuyo número de tarjeta ha sido validado previamente en la tabla de tarjetas) registrar un pago para un comercio en particular y una transacción concreta.

3. `testbd/`: Permite realizar pruebas en la base de datos. Combina funcionalidades de verificación de tarjetas en la tabla de tarjetas y de registro de pagos, facilitando la integración de ambas operaciones en un entorno de pruebas. Igualmente permite borrar y listar los pagos asociados a un cierto comercio.

Ejercicio 1: incluya en la memoria de la practica pruebas de que se ha desplegado la aplicación web localmente y de que esta funciona. Al menos se debe incluir una copia del contenido del fichero **env** así como una captura de pantalla en la que se muestre el resultado de interaccionar con cada uno de los "endpoints". Las capturas deben mostrar el URL al que se conecta el navegador.

Ejercicio 2: ejecuta los test proporcionados con el proyecto base y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test (`python manage.py test`). Los test proporcionados deben tomarse como requisitos extras del sistema.

4. Despliegue de la aplicación en las máquinas virtuales

Como se comentó en la introducción de la práctica deseamos desplegar la aplicación en la máquina virtual **VM2** manteniendo la base de datos en **VM1** (véase figura 3).

4.1. Requisitos Previos

En el PC del laboratorio o máquina host

Antes de proceder, asegúrate de tener:

- acceso SSH a las máquinas virtuales en los puertos `12022` (**VM1**) y `22022` (**VM2**).
- *git* instalado en la máquina local.

En la máquina VM1

- crea la base de datos **si2db**.

- inicialízala desde el **host** modificando la variable **DATABASE_SERVER_URL** en el fichero **env** y ejecutando `python manage.py migrate`

En la máquina VM2

- crea un repositorio *git* tal y como se describe en el apéndice B.
- configura la máquina para que la aplicación web se sirva usando **gunicorn** y para que este se relance cada vez que se modifique el código (véase apéndice C).
- actualiza el fichero env para que acceda a la base de datos situada en **VM1**. Recuerda que por razones de seguridad el fichero `env` no debe añadirse al repositorio.
- añade los siguientes comandos al fichero `/home/si2/repo/p1base.git/hooks/post-receive`

```
python $TARGET/manage.py migrate
python $TARGET/manage.py collectstatic --noinput
```

  - **python $TARGET/manage.py migrate**: Aplica los cambios pendientes en la base de datos basándose en las definiciones de los modelos, asegurando que su estructura esté sincronizada con el código del proyecto.
  - **python $TARGET/manage.py collectstatic –noinput**: Recopila los archivos estáticos (CSS, JavaScript, imágenes) en un único directorio central (`STATIC_ROOT`) para ser servidos en entornos de producción.

Ejercicio 3: incluya en la memoria de la practica pruebas de que se ha desplegado la aplicación web **P1-base** usando las máquinas virtuales y de que esta funciona. Al menos se debe incluir una copia del contenido del fichero **env** así como una captura de pantalla en la que se muestre el resultado de registrar un pago. Esta última debe mostrar claramente el URL al que se conecta el navegador.

Ejercicio 4: El script de python incluido en el apéndice D lee 1000 entradas de la base de datos "si2db" una a una e imprime el tiempo de lectura. Ejecútalo desde el ordenador **host** y reporta en la memoria de la práctica los siguientes casos:

1. La base de datos está en la máquina virtual **VM1**.
2. La base de datos no esta en la red local. Crea un base de datos llamada `si2db` en `http://www.neon.tech` y ejecuta el script contra esa base de datos
3. No se accede a la base de datos directamente. En Django, el mapeo de clases a tablas se implementa mediante el ORM (Object-Relational Mapping). Reescribe el script para que las tarjetas se lean usando el sistema de modelos de *Django*. Por ejemplo `'SELECT * FROM tarjeta WHERE ‘‘numero’’ =%s'` debería ser `Tarjeta.objects.get(pk=%s)`. En este nuevo script no debe aparecer código SQL). Para ver cómo usar el modelo de datos de Django desde el script, revisar el Apéndice A.

Repite cada medida 7 veces y reporta el valor medio de las mismas así como su desviación estándar. Además de los tiempos de lectura incluye en la memoria el script reescrito, el contenido del fichero **env** en cada caso así como un comentario sobre los resultados obtenidos. ¿Usar el ORM de Django implica alguna sobrecarga?

Cuestion 1: ¿En el ejercicio anterior 7 repeticiones de cada medida proporcionan una estimación fiable? ¿Por qué? Justifica tu respuesta con los datos obtenidos.

Ejercicio 5: ejecuta los test proporcionados con el proyecto **P1-base** y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test incluyendo el comando utilizado para lanzarlos. En está ejecución la base de datos debe estar en **VM1** y la aplicación web en **VM2**.

5. Introducción a la API REST

En esta sección veremos como podemos dividir la aplicación de partida **P1-base** en dos partes remotas (frontend y backend) que se comuniquen mediante servicios web basados en REST. El esquema a seguir sería el mostrado en la figura 4.

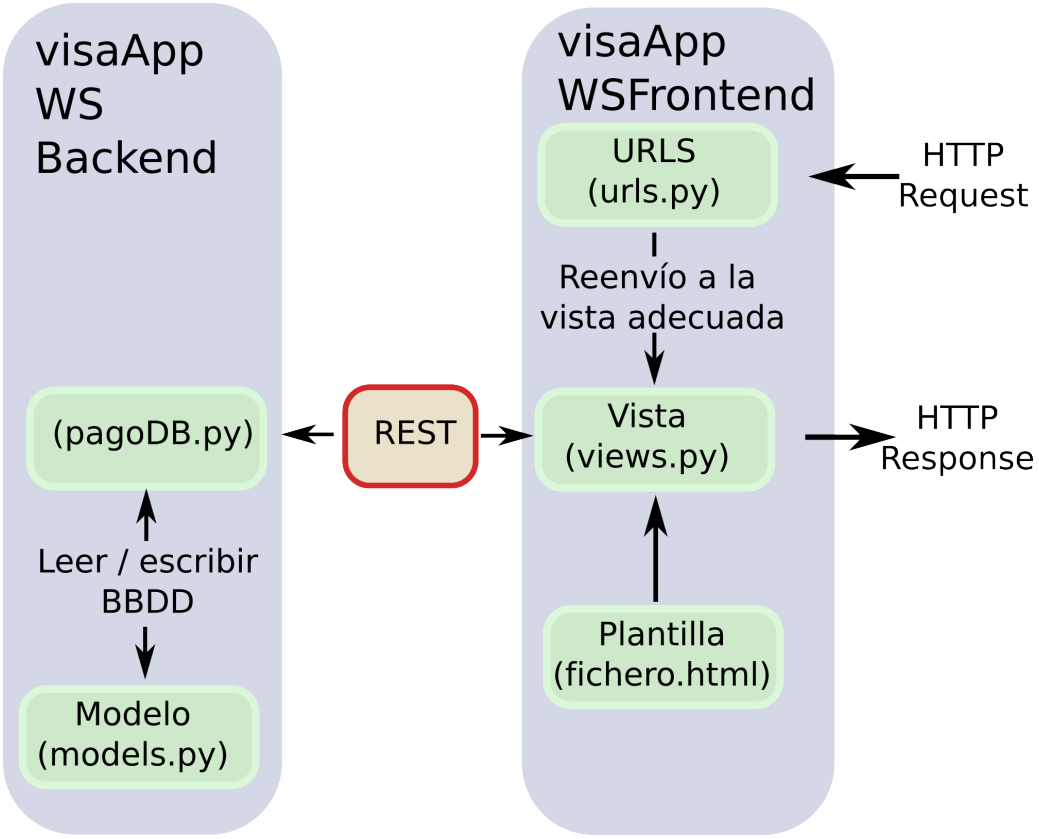

Figura 4: Descripción esquemática del flujo de información de la aplicación P1-base modificada para separarla en 2 partes (frontend y backend) donde se lleve a cabo la comunicación mediante servicios web de tipo REST.

Una **API REST** (*Representational State Transfer*) es una interfaz que permite la comunicación entre aplicaciones mediante el intercambio de datos, siguiendo los principios y convenciones del protocolo HTTP. REST es una arquitectura ampliamente adoptada debido a su simplicidad, escalabilidad y compatibilidad con diferentes sis-

temas.

5.1. Características de una API REST

- **Recursos**: Los datos se exponen como recursos accesibles a través de URLs únicas.
- **Operaciones estándar**: Se utilizan métodos HTTP como `GET` (listado de modelos y detalles de un modelo concreto), `POST` (creación de un modelo), `PUT` (actualización de un modelo) y `DELETE` (borrado de un modelo) para interactuar con los recursos.
- **Formato de datos**: Los datos suelen intercambiarse usando `JSON` o formatos equivalentes.
- **Stateless**: Cada solicitud a la API es independiente, lo que significa que no se almacena el estado de las interacciones en el servidor.

En esta sección implementaremos primero el backend (servidor) API REST y posteriormente el frontend (cliente) que consumirá este API.

5.2. Implementación del API REST usando *Django*

Primero describiremos la funcionalidad de la API REST y tras acabar con la descripción de los diferentes endpoints ofreceremos algunas ideas sobre como implementarlos. La API REST solicitada se desarrollará usando el `Django REST Framework` (DRF) con los siguientes "endpoints" principales.

- Validación de una tarjeta en la base de datos.
- Registro de un pago en comercios específicos.
- Consulta (listado de pagos emitidos para comercio dado) y eliminación de pagos por ID.

Los "endpoints" se definen mediante el fichero `urls.py`. Para que los test funcionen correctamente se debe conservar las etiquetas "name='xxxx' ". A continuación se muestra el contenido que debe tener dicho fichero:

```
urlpatterns = [
   # check if tarjeta is in "tarjeta"
   path('tarjeta/', TarjetaView.as_view(),
      ↪ name='tarjeta'),
   # create "pago"
   path('pago/', PagoView.as_view(), name='pago'),
   # get list of "pagos" associated with a given
      ↪ idComercio
   path('comercio/<str:idComercio>',
        ComercioView.as_view(), name='comercio'),
   # delete "pago" with id id_pago
   path('pago/<str:id_pago>', PagoView.as_view(),
      ↪ name='pago'),
   ]
```

donde TarjetaView, PagoView y ComercioView son objetos a implementar usando la API rest de Django, como se explica más adelante. Esto hará que los URLs a los que se conectará el frontend o cliente sean del tipo `http://<host_name>/restapiserver/<endpoint>/`.

- La validación de la existencia de la tarjeta en la tabla de tarjetas se realiza en el "endpoint" `tarjeta` al cual se accederá mediante una llamada de tipo `POST` que contendrá la información de la tarjeta. La llamada debe devolver el estado **HTTP_200_OK** en caso de éxito (la tarjeta existe) o **HTTP_404_NOT_FOUND** en caso de que la tarjeta no exista en la tabla de tarjetas. Igualmente debe devolver un diccionario con un mensaje bien sea de éxito o de fracaso. Por ejemplo

```
return Response(
        {'message': 'Datos de la tarjeta no
           ↪ encontrados.'},
        status=status.HTTP_404_NOT_FOUND)
```

- De forma similar, el registro del pago se realiza en el endpoint `pago` al cual se accederá mediante una llamada de tipo `POST` que contendrá la información del pago. Esta información debe contener el identificador de la tarjeta asociada al pago (su número), al igual que ocurría en la aplicación visaApp. El API devolverá el estado **HTTP_200_OK** en caso de éxito, **HTTP_404_NOT_FOUND** si la tarjeta no existe en la base de datos o **HTTP_400_BAD_REQUEST** en el resto de los casos. Igualmente devolverá el pago registrado en caso de éxito y un mensaje de error en el resto de los casos. Por ejemplo:

```
if pago is None:
    return Response({'message': 'Error al registrar
        ↪ pago.'},
            status=status.HTTP_404_NOT_FOUND)
pago_dict = model_to_dict(pago)
return Response(pago_dict, status=status.HTTP_200_OK)
```

- Para implementar las consultas (listado de pagos de un comercio) se utilizará una llamada `GET` a `comercio` en el que se pasará el identificador del comercio para el cual se solicita el listado de pagos. El API devolverá el estado **HTTP_200_OK** en caso de existir pagos asociados al comercio solicitado o **HTTP_404_NOT_FOUND** si no existen. Igualmente devolverá el listado de pagos en caso de éxito y un mensaje de error en el resto de los casos.

- Finalmente para eliminar un pago se realizará una llamada `DELETE` a `pago` pasando el identificador del pago que se desea borrar. El API devolverá el estado **HTTP_200_OK** en caso de haberse borrado el pago y **HTTP_404_NOT_FOUND** si el pago no existe y por tanto no es posible borrarlo.

Se solicita que implementéis el API REST usando clases que hereden de `APIView` tal y como se describe en `https://davidcasr.medium.com/construir-un-api-rest-con-django-rest-framework-y-apiview-5ea4b2823307` en el paso quinto y siguientes. Recordad que las operaciones para llevar a cabo el registro de pago, su borrado, listado, frente a la base de datos están disponibles en el fichero pagoDB.py **y estas deberán ser utilizadas por las `APIViews` implementadas**.

El primer paso para crear esta nueva versión de nuestro web site será crear un nuevo proyecto, correspondiente al backend, partiendo del proyecto P1-base. Para ello, copiaremos el directorio P1-base a **P1-ws-backend** con el comando:

```
cp -r P1-base P1-ws-backend
```

El siguiente paso, será renombrar la aplicación, que en P1-base se llamaba visaApp a visaAppWSBackend. Para ello, ejecutaremos los comandos:

```
cd P1-ws-backend
mv visaApp visaAppWSBackend
find ./ -type f -exec sed -i "s/visaApp/visaAppWSBackend/g" {} \;
```

El siguiente paso será eliminar los ficheros no necesarios en la parte del backend (servidor). En particular, aquellos relacionados con la interfaz web. Para ello ejecutaremos estos comandos:

```
rm  visaAppWSBackend/forms.py
# views.py no se necesita
rm  visaAppWSBackend/views.py
rm -rf visaAppWSBackend/templates
# sobreescribe test_views.py con el fichero test_views.py
# existente en el repositorio para este proyecto.
rm visaAppWSBackend/tests_views.py
rm visaAppWSBackend/tests_models.py
```

El fichero `views.py` necesita ser reescrito puesto que ahora algunas funciones se crean de forma automática y el resto en la versión original devuelve código HTML y ahora debe devolver JSON. Una vez que esté implementado el API y el servidor esté arrancado al conectaros al mismo (p.e. `http://<hostname>:8000/restapiserver/pago/`) usando el navegador deberíais obtener un resultado similar a:

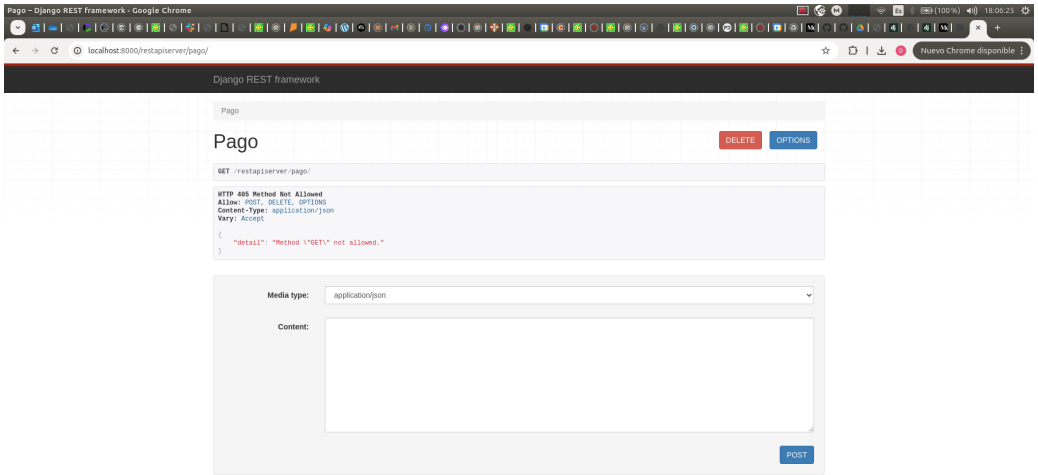

Figura 5: Pantalla mostrando el resultado de conectarse al URL http://<hostname>:8000/restapiserver/pago/ donde se ofrecen conexiones de tipo `POST` (crear) o `DELETE` (borrar) y se deniegan las de tipo `GET` (ver detalles de los pagos). En este ejemplo el servidor está siendo ejecutado en el ordenador **host** sirviendo en el puerto 8000.

Como en el caso anterior despliega el backend (servidor) en **VM2** y mantén la base de datos en **VM1**.

Ejercicio 6: ejecuta los test proporcionados con el proyecto **P1-ws-backend** aplicación **visaAppWSBackend** y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test junto con el comando utilizado. En está ejecución la base de datos debe estar en **VM1** y la aplicación web en **VM2**.

Ejercicio 7: conéctate a los diferentes endpoints usando el navegador (http://<hostname>:28000/restapiserver/xxxx/) y muestra las pantallas resultantes de usar cada uno de ellos. Suministra al sistema datos válidos e inválidos. Junto a cada imagen debes incluir: (1) el URL al que te conectas, (2) el método usado (`POST/GET/DELETE`) y (3) en caso de usar el método `POST` los datos enviados en la petición

5.3. Implementación del Frontend (Cliente) del API REST usando *Django*

El siguiente paso será implementar el frontend que consuma nuestro API REST. Para implementar el sistema entero (frontend-backend) usaremos 2 PCs y un total de tres máquinas virtuales tal y como se describe en la figura 6.

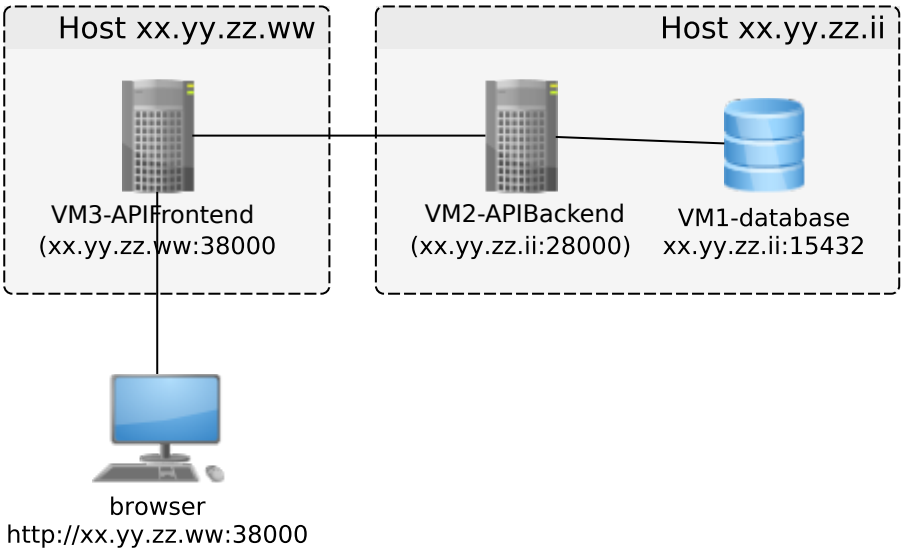

Figura 6: Descripción esquemática del despliegue de las aplicaciones `P1-ws-backend` y `P1-ws-frontend` **P1-base**.

Crearemos un nuevo proyecto llamado `P1-ws-frontend` el cual contendrá la apli-

cación `visaAppWSFrontend` la cual usará los mismos modelos y el mismo fichero `urls.py` que se definió en el proyecto `P1-base`. En esta implementación el navegador llamará al proyecto `P1-ws-frontend`, este proyecto sólo puede acceder a la base de datos haciendo peticiones al API REST implementado en el proyecto `P1-ws-backend`. Una manera sencilla de implementar este proyecto será, como en el caso anterior, duplicar el proyecto `P1-base` como `P1-ws-frontend`. A continuación de borraran los ficheros:

```
rm -rf visaAppWSFrontend/models.py
rm -rf visaAppWSFrontend/migrations
rm -rf visaAppWSFrontend/management
rm -rf visaAppWSFrontend/tests_models.py
# sobreescribe test_views.py con el fichero test_views.py
# existente en el repositorio para este proyecto.
```

A continuación habrá que **modificar el fichero** `pagoDB.py` para que llame al API REST en lugar de acceder a la base de datos directamente a través de los modelos de *Django*. Para conseguir este objetivo hay que realizar llamadas de tipo `response = requests.post(api_url, json=pago_dict)` en lugar de `pago = Pago.objects.create(**pago_dict)`. En este ejemplo se realiza una conexión al URL `api_url` al cual se le pasa un diccionario con la información relacionada con el pago `pago_dict` que se desea crear. Si la respuesta contiene información en formato json, por ejemplo, del pago modificado, se puede obtener de la forma `response.json()`.

Igualmente es necesario añadir una nueva variable llamada `RESTAPIBASEURL` conteniendo la dirección en la que el API REST está escuchado (p.e. `http://localhost:8000/restapiserver/`) al archivo `env`. Esta variable debe ser leída por el fichero `settings.py` de forma que sea accesible en el resto del proyecto.

No será mofiicar el fichero views.py del Frontend, que puede ser exactamente el mismo que se utilizaba en P1-base. Solo será necesario modificar pagoDB.py.

Ejercicio 8: ejecuta los test proporcionados con el proyecto **P1-ws-frontend** aplicación **visaAppWSFrontend** y comprueba que no devuelven errores. Adjunta en la memoria una captura de pantalla en la que se muestre el resultado de ejecutar los test. En está ejecución la base de datos debe estar en **VM1** y la aplicación web en **VM2**.

Ejercicio 9: incluya en la memoria de la practica pruebas de que se ha desplegado las aplicaciones web `P1-ws-frontend` y `P1-ws-backend` y de que estas funcionan. Al menos se debe incluir una copia del contenido del fichero **env** así como una captura de pantalla en la que se muestre el resultado de interaccionar con cada uno de los "endpoints" del frontend. Las capturas deben mostrar el URL al que se conecta el navegador.

Material a entregar al finalizar la práctica: se debe subir a *Moodle* un fichero zip que contenga la memoria (en formato pdf) respondiendo a todos los ejercicios y cuestiones planteadas en esta guía junto con el código utilizada para implementar los diferentes proyectos. Se considerará que el código funciona si lo hace en los ordenadores del laboratorio. El código entregado en *Moodle* será el evaluado, bajo ninguna condición se evaluará el código existente en los diversos repositorios.

6. Criterios de evaluación

Para aprobar es necesario haber implementado el API REST y satisfacer los ejercicios 6 y 7. En caso de satisfacer este requerimiento se optiene una nota de 5 a la que se sumarán las cantidades siguientes:

Ejercicio	Puntuacion
1	0.5
2	0.5
3	0.5
4	0.5
5	0.5
8	1.0
9	1.0
Cuestion	
1	0.5

A. Configuración de Django

Para poder acceder al modelo de datos de Django es necesario inicializarlo previamente. Para ello, se puede usar el código:

```
# Configurar Django
import os, django
os.environ.setdefault("DJANGO_SETTINGS_MODULE", "visaSite.settings")
django.setup()
```

B. Configuración de un Repositorio Git Remoto Usando SSH

Este apéndice explica cómo crear un repositorio Git remoto en otro equipo mediante SSH y habilitar el uso de `git push`.

B.1. Crear un Repositorio Git Bare en el Equipo Remoto VM2

1. Conéctate al equipo remoto mediante SSH:

```
# host
# el puerto (22022) depende de la maquina a la que desees acceder
ssh si2@localhost -p 22022
```

2. Crea un repositorio bare (sin árbol de trabajo):

```
# MV2
mkdir -p /home/si2/repo
cd /home/si2/repo
git init --bare p1base.git -b main
```

El último comando debe devolver el mensage `Initialized empty Git repository in /home/si2/repo/p1base.git/`

B.2. Conectar el Repositorio Local al Remoto

1. Navega al directorio de tu proyecto en el **host** el cual ya es un repositorio de git.

2. Agrega el repositorio remoto **p1base.git** al listado de repositorios remotos con el alias **vm2**:

```
# host
git remote add vm2
    ↪ ssh://si2@localhost:22022/~/repo/p1base.git
```

3. Verifica que el remoto se agregó correctamente:

```
# host
git remote -v
```

B.3. Actualizar Archivos Automáticamente en el Equipo Remoto

Cuando ejecutas el comando `git push`, estás transfiriendo los **commits** desde el repositorio local hacia el repositorio remoto. Sin embargo, esto **no actualiza directamente los archivos de trabajo en el servidor remoto**, únicamente la base de datos de *git* se actualiza. En algunos casos, como cuando usas *git* para desplegar un sitio web, el repositorio remoto puede estar asociado a un área de trabajo. Para que los archivos de trabajo reflejen los cambios más recientes, deberás configurar el repositorio remote como se describe a continuación.

1. En **VM2** crea o edita el archivo `post-receive` en el repositorio bare del equipo remoto). Como ya sospecharás el fichero `post-receive` se ejecuta cada vez que la máquina recibe un **git push**:

```
#VM2
nano /home/si2/repo/p1base.git/hooks/post-receive
```

2. Agrega las siguientes líneas al archivo:

```
#!/bin/bash
GIT_WORK_TREE=/home/si2/repo/p1base git checkout -f
```

   Si el directorio `/home/si2/repo/p1base` **no existe créalo**. Esta será la ubicación del directorio de trabajo.

3. Haz que el archivo sea ejecutable:

```
chmod +x /home/si2/repo/p1base.git/hooks/post-receive
```

4. Finalmente en el **host** ya puedes ejecutar el comando **git push**.

```
# host
git push -u vm2 main
```

y el código se actualizará con la última versión.

B.4. Realizar un Push Desde el Repositorio Local

1. Los cambios se envían al repositorio remoto con el comando:

```
git push -u vm2 main
                        # -u establece esta
                          ↪ configuracion por defecto
                          ↪ de forma que
                        # de ahora en adelante no es
                          ↪ necesario teclear "-u vm2
                          ↪ main"
```

C. Configuración de gunicorn

Cuando se despliega la aplicación web en modo "producción" no se utiliza el servidor de desarrollo que acompaña a *Django* sino un contenedor de aplicaciones como **gunicorn**.

Para que **gunicorn** se relance cada vez que se despliega código nuevo debemos modificar el archivo `post-receive` en **VM2** y añadir la línea

```
# Reiniciar Gunicorn
sudo systemctl restart gunicorn
```

Para que el comando `systemctl restart gunicorn` funcione correctamente en el contexto de un "hook" de *git*, es importante configurar y asegurarte de que el servicio **gunicorn** esté gestionado adecuadamente por **systemd**. Aquí está una guía completa para configurarlo.

C.1. Crear un Archivo de Configuración para el Servicio Gunicorn

Crea un archivo de configuración para **gunicorn** en el directorio `/etc/systemd/system/` de **VM2**:

```
sudo nano /etc/systemd/system/gunicorn.service
```

Agrega la configuración del servicio. Aquí hay un ejemplo para una aplicación basada en *Django*:

```
[Unit]
Description=Gunicorn WSGI Application Server
After=network.target

[Service]
User=si2
Group=si2
WorkingDirectory=/home/si2/repo/<nombre-del-proyecto>
```

```
Environment="PATH=/home/si2/venv/bin"
ExecStart=/home/si2/venv/bin/gunicorn \
          --workers 1 \
          --bind 0.0.0.0:8000 \
          visaSite.wsgi:application
ExecReload=/bin/kill -s HUP $MAINPID
KillMode=mixed
TimeoutStopSec=5
PrivateTmp=true

[Install]
WantedBy=multi-user.target
```

C.2. Explicación de los Parámetros Importantes

- `User` y `Group`: Define al usuario y grupo que ejecutarán **gunicorn**. Normalmente se utiliza `www-data` aunque aquí usemos **si2**.
- `WorkingDirectory`: Ruta al directorio raíz de tu proyecto
- `ExecStart`: El comando para iniciar Gunicorn. Incluye:
  - Ruta al binario de **gunicorn** dentro del entorno virtual.
  - `-workers`: Número de procesos de trabajadores.
  - `-bind 0.0.0.0:8000`: escucha en el puerto 8000 y admite conexiones desde cualquier host.
  - `visaSite.wsgi:application`: Especifica la entrada WSGI de tu aplicación.

C.3. Recargar `systemd` y Activar gunicorn

Después de crear el archivo, recarga `systemd` para aplicar los cambios:

```
sudo systemctl daemon-reload
```

Habilita **gunicorn** para que se inicie automáticamente con el sistema:

```
sudo systemctl enable gunicorn
```

Inicia el servicio **gunicorn**:

```
sudo systemctl start gunicorn
```

Verifica que **gunicorn** esté funcionando:

```
sudo systemctl status gunicorn
```

D. Script que lee la tabla tarjeta

Script en Python que leer las primeras 1000 entradas de una tabla llamada tarjeta en una base de datos llamada si2db y muestra el tiempo invertido en la lectura:

Listado 2: código del script read_1000_entries_from_db.py

```
# connect to the database, read the first 1000 entries
# then perform 1000 queries retrieving each one of the entries
# one by one. Measure the time requiered for the 1000 queries

import psycopg2
import time

# Configuracion de la base de datos
db_config = {
    'dbname': 'si2db',  # Nombre de la base de datos
    'user': 'alumnodb',  # Reemplaza con tu usuario de PostgreSQL
    'password': 'alumnodb',  # Reemplaza con tu contrasegna
    'host': 'localhost',  # Cambia si el host es diferente
    'port': 5432,  # Cambia si tu puerto es diferente
```

```
}

try:
    # Conexion a la base de datos
    conn = psycopg2.connect(**db_config)
    cursor = conn.cursor()

    # Leer las primeras 1000 entradas de la tabla censo
    query_fetch_1000 = "SELECT * FROM tarjeta LIMIT 1000"
    cursor.execute(query_fetch_1000)
    rows = cursor.fetchall()

    # Preparar para las busquedas individuales
    search_query = 'SELECT * FROM tarjeta WHERE \"numero\" = %s'  # Asumiendo que hay una columna 'id' para identificar las filas

    # Medir el tiempo de inicio
    start_time = time.time()

    # Realizar busquedas una a una
    for row in rows:
        id_value = row[0]  # Suponiendo que la primera columna es el ID
        cursor.execute(search_query, (id_value,))
        cursor.fetchone()  # Obtener la fila encontrada

    # Medir el tiempo de finalizacion
    end_time = time.time()

    # Mostrar los resultados
    print(f"Tiempo invertido en buscar las 1000 entradas una a una: {end_time - start_time:.6f} segundos")
```

```

except Exception as e:
    print(f"Error: {e}")

finally:
    # Cerrar el cursor y la conexion
    if 'cursor' in locals():
        cursor.close()
    if 'conn' in locals():
        conn.close()
```

Código disponible en este enlace: